

Frontend Developer Guide

Complete reference for engineers contributing to **inventory-reservation-client** — a React 18 + TypeScript 5 single-page application built on Domain-Driven Design, SOLID principles, and Event-Driven Architecture.

• React 18.3

• TypeScript 5.5 strict

• Vite 6

• [DDD](#) · [SOLID](#) · [EDA](#)

• Zero runtime deps beyond React

Quick Start

Prerequisites

TOOL	VERSION	NOTES
nvm	—	Node Version Manager — install from nvm-sh/nvm
Node.js	22	Switched via <code>nvm use 22</code>
npm	bundled with Node 22	No Yarn / pnpm needed
NestJS backend	—	Must be running on <code>http://localhost:3000</code> before use

Install & Run

```
# 1. Switch to Node 22 (install if not present)
nvm install 22
nvm use 22

# 2. Install dependencies
npm install

# 3. Start dev server (hot-reload at http://localhost:5173)
npm run dev

# 4. Production build — must pass before any commit
```

```
npm run build
```

```
# 5. Preview the built output
```

```
npm run preview
```

First-time setup

On first load the app reads `api_base` from `localStorage` (defaults to `http://localhost:3000`). Change it anytime in the **Settings** tab — no restart required.



Seed inventory before creating reservations

The backend's Redis stock counter must be primed first. Open **Settings** → **Seed Inventory** and add at least one product. A 409 Conflict on reservation creation means inventory is unseeded.

Tech Stack

CONCERN	CHOICE	WHY
UI framework	React 18.3	Concurrent rendering, hooks-first API
Language	TypeScript 5.5 strict	<code>noImplicitAny</code> , <code>strictNullChecks</code> , <code>noUnusedLocals</code> all on
Bundler	Vite 6	Sub-second HMR; native ESM dev server
Styling	CSS custom properties + inline styles	Zero dependency; Obsidian dark theme; no Tailwind, no CSS-in-JS
State	React Context + <code>useState</code>	No Redux/Zustand — app state is shallow; Context is sufficient
HTTP	Custom <code>HttpClient</code> wrapping <code>fetch</code>	No Axios; dynamic base URL from <code>localStorage</code> ; lean bundle
Routing	Tab model in <code>AppContext</code>	No React Router — five tabs, no deep links needed
Testing	None yet	Repository interfaces designed for easy in-memory substitution when tests are added



Zero runtime deps beyond React

`package.json` dependencies contains only `react` and `react-dom`. UUID generation, pub/sub, DI wiring, and HTTP are all hand-rolled in `src/`. This demonstrates the patterns without library magic and keeps the bundle under 200 kB.

Layer Map

The codebase is divided into six layers. **Dependencies only point inward** — outer layers may import from inner layers, but the reverse is never allowed. This enforces the Dependency Inversion Principle at the architectural level.



container.ts (DI wiring)



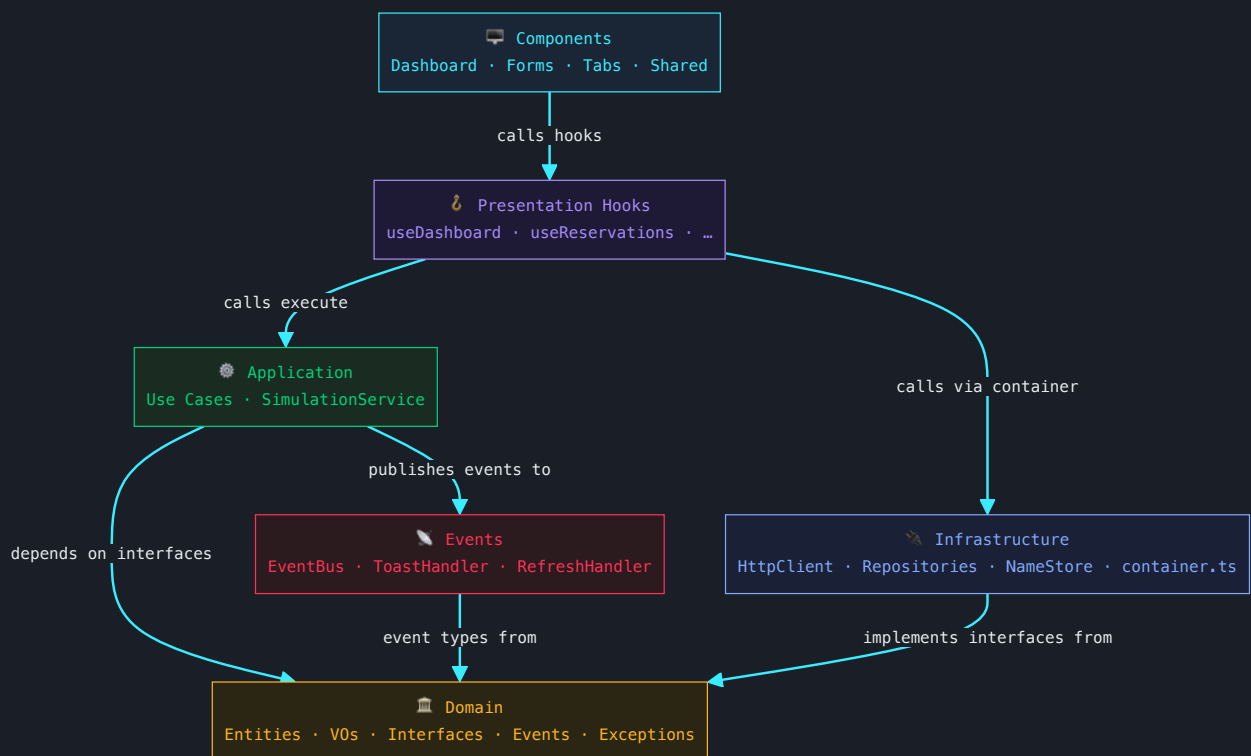
The dependency rule

Components → Hooks → Use Cases → Domain interfaces. Infrastructure implements those interfaces from the outside. Nothing in Domain or Application ever imports from Infrastructure, Components, or the Events layer. Violations break DIP and must not be introduced.

Diagram — Layer Dependency Graph

Every arrow points inward. The Domain layer imports nothing — it is the stable core everything else depends on.

ARROWS = "DEPENDS ON / IMPORTS FROM"



Data Flow

Read path — loading the dashboard

Dashboard

useDashboard

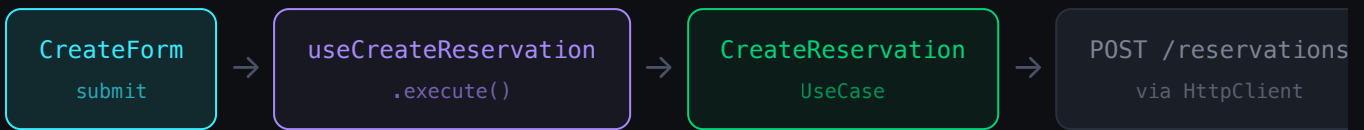
GetDashboardStats

IReservationRepo

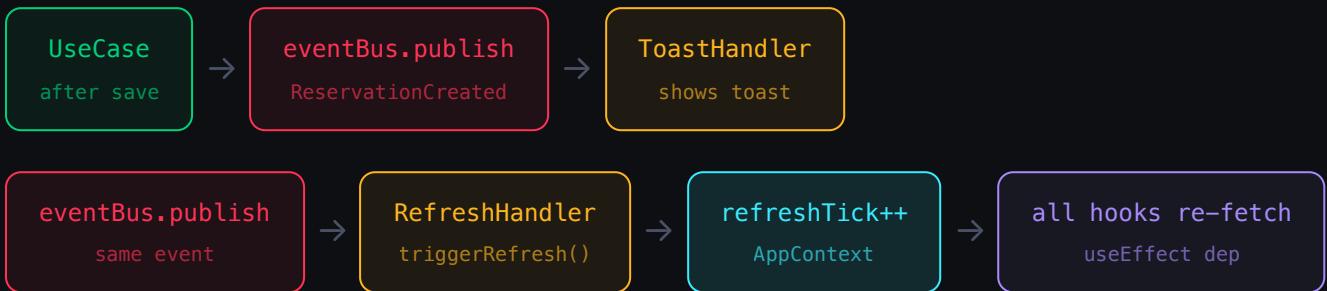
Ap:



Write path — creating a reservation



Event path — after any mutation

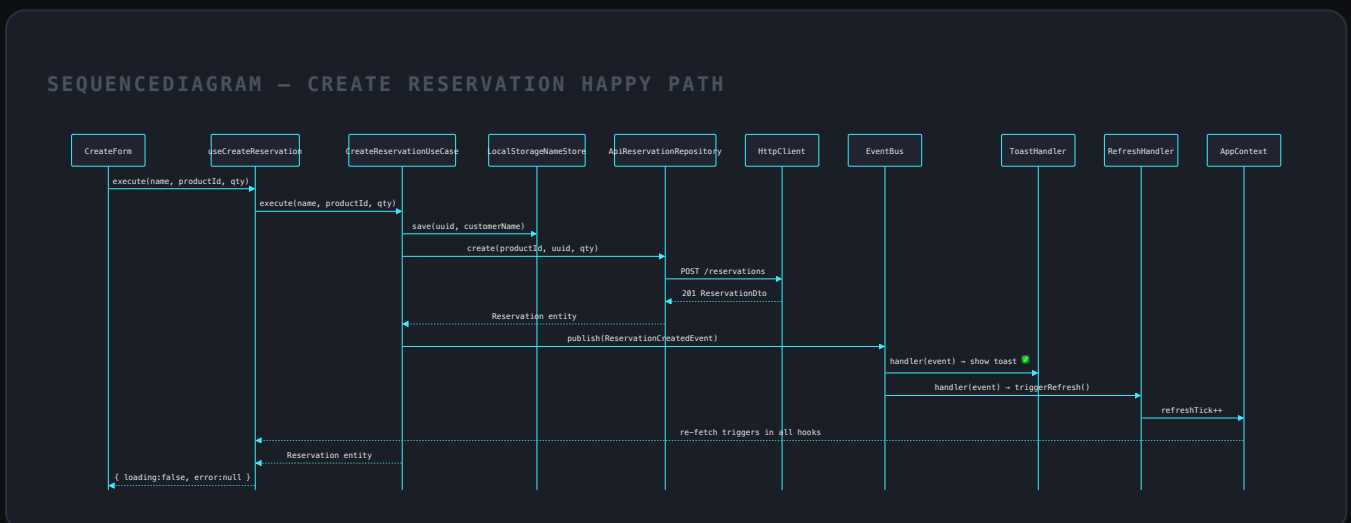


✓ Components never call fetch directly

All network activity flows: hook → use case → repository → HttpClient. Components render state returned by hooks and call hook action functions on user interaction. This means any component can be integration-tested by replacing its hook with a stub.

Diagram — Full Write Request Sequence

End-to-end trace of **Create Reservation** showing how all layers collaborate and how the event bus decouples toast + refresh from the use case.



File Tree

```
src/
├─ main.tsx           React root, mounts <AppProvider>
├─ App.tsx            Tab router – conditional render, no library
├─ index.css          ALL CSS: tokens, globals, utility classes
├─
├─ types/
│   └─ index.ts       TabId union + other shared types
├─
├─ context/
│   └─ AppContext.tsx Global state + EDA handler wiring
├─
├─ domain/           Pure business logic – zero external imports
│   └─ entities/
│       ├── Reservation.ts Rich entity: isActive, canConfirm, expiryMath
│       └─ Inventory.ts    availableStock, isOutOfStock, isLowStock
│   └─ value-objects/
│       └─ ReservationStatus.ts ACTIVE | CONFIRMED | CANCELLED | EXPIRED + display m
│   └─ interfaces/
│       ├── IReservationRepository.ts
│       ├── IInventoryRepository.ts
│       └─ INameStore.ts
│   └─ events/
│       ├── DomainEvent.ts
│       └─ index.ts       4 event classes
│   └─ exceptions/
│       └─ index.ts       OutOfStockError, InvalidStateError, NotFoundError
├─
├─ application/
│   └─ use-cases/
│       ├── CreateReservationUseCase.ts
│       ├── ConfirmReservationUseCase.ts
│       ├── CancelReservationUseCase.ts
│       ├── GetReservationsUseCase.ts
│       ├── GetDashboardStatsUseCase.ts
│       └─ SeedInventoryUseCase.ts
│   └─ services/
│       └─ SimulationService.ts 6 simulation scenarios
│   └─ utils/
│       └─ uuid.ts
```

```
|
|
|— events/                               Pub/sub bus and reaction handlers
|   |— EventBus.ts                       Singleton – subscribe() returns unsubscribe fn
|   |— handlers/
|       |— ToastHandler.ts               Shows toasts on domain events
|       |— RefreshHandler.ts            Calls triggerRefresh() on mutating events
|
|
|— infrastructure/                       Implements domain interfaces – knows about HTTP
|   |— http/
|       |— HttpClient.ts
|   |— repositories/
|       |— ApiReservationRepository.ts
|       |— ApiInventoryRepository.ts
|   |— stores/
|       |— LocalStorageNameStore.ts
|   |— container.ts                      DI wiring – ONLY file importing both interfaces & impl
|
|
|— presentation/
|   |— hooks/                            Bridge use cases to React state
|       |— useDashboard.ts
|       |— useReservations.ts
|       |— useCreateReservation.ts
|       |— useSimulate.ts
|       |— useInventory.ts
|
|
|— components/
|   |— Layout/
|       |— Sidebar.tsx                   Nav, LIVE/OFFLINE dot, version footer
|       |— TopBar.tsx                   Breadcrumb, live clock, refresh button
|   |— shared/
|       |— Badge.tsx
|       |— ExpiryBar.tsx                 Accepts Reservation entity, ticks every 1s
|       |— Modal.tsx
|       |— Spinner.tsx
|       |— Toast.tsx
|   |— Dashboard/
|   |— Reservations/
|   |— CreateForm/
|   |— Simulate/
|   |— Settings/
```

Domain Layer

The domain layer contains pure TypeScript business rules. **No React, no fetch, no localStorage.** Every other layer may import from here; this layer imports from nowhere. This is what makes the business logic portable, testable, and framework-agnostic.

Entities

RESERVATION

A rich entity that owns its own business logic. Components never recompute status or expiry — they call entity methods.

```
class Reservation {
  // Constructor args – all readonly
  id: string; productId: string; productName: string
  userId: string; quantity: number; status: ReservationStatus
  expiresAt: Date; createdAt: Date

  // State queries
  get isActive(): boolean
  get isConfirmed(): boolean
  get isCancelled(): boolean
  get isExpired(): boolean

  // Business rules
  get canConfirm(): boolean // true only when ACTIVE
  get canCancel(): boolean // true only when ACTIVE

  // Expiry calculations – used by ExpiryBar
  get expiryPercentElapsed(): number // 0–100
  get timeRemainingMs(): number // 0 when expired

  // Immutable status update
  withStatus(status: ReservationStatus): Reservation
}
```

INVENTORY

```
class Inventory {
  productId: string; name: string; totalStock: number
  confirmedSales: number; activeReservations: number; updatedAt: Date

  get availableStock(): number // totalStock – confirmedSales – activeReservations
}
```



```

get isOutOfStock(): boolean
get isLowStock(): boolean      // availableStock < 5
}

```

Value Object — ReservationStatus

```

type ReservationStatus = 'ACTIVE' | 'CONFIRMED' | 'CANCELLED' | 'EXPIRED'

// Display metadata used by Badge and status rendering
ReservationStatusMeta[status] → { label: string, colorVar: string, pulsing: boolean

```

Diagram — Reservation Lifecycle

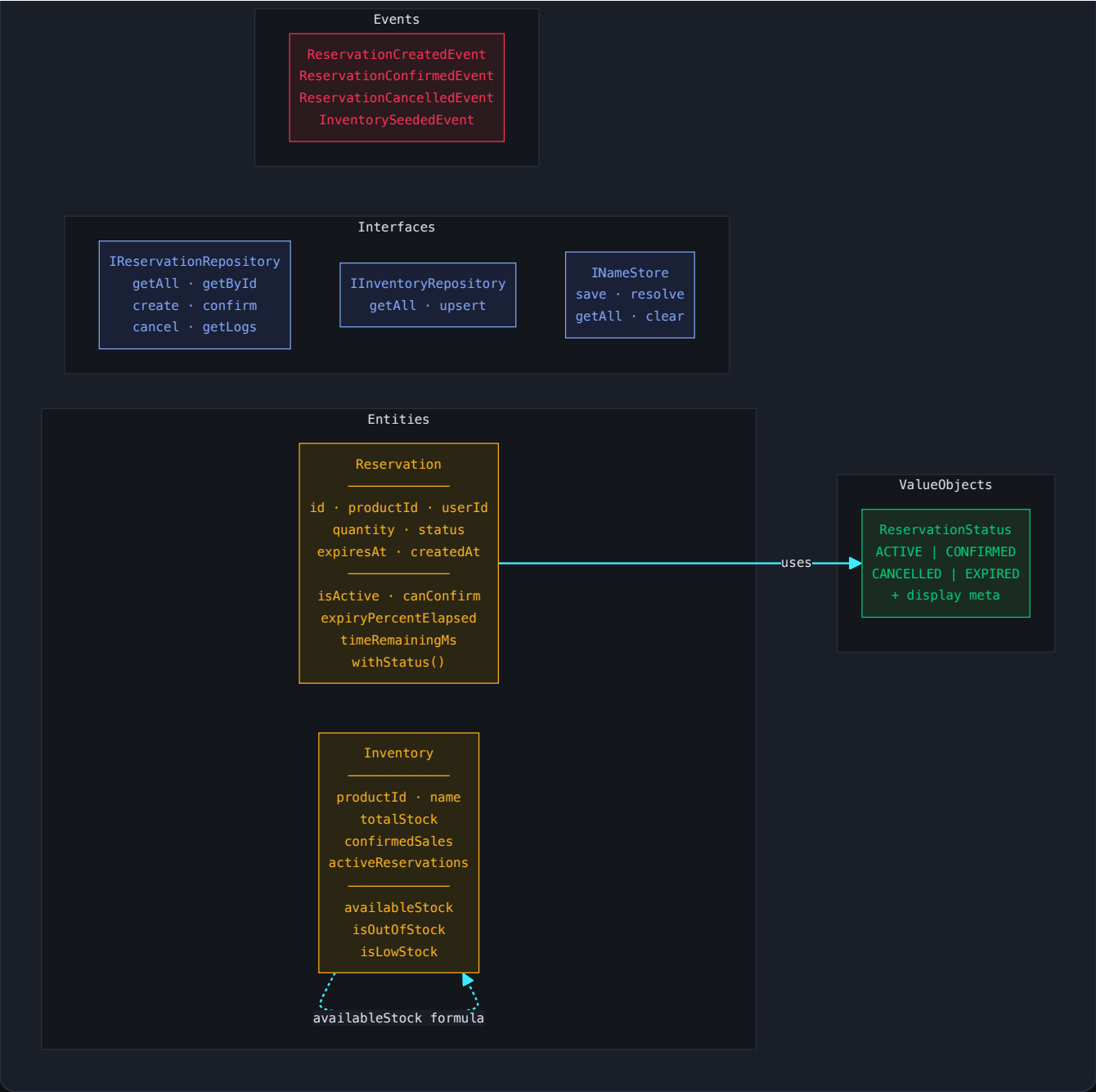
The `Reservation` entity owns all transition logic. `canConfirm` and `canCancel` are computed getters — components never duplicate this check.

STATEDIAGRAM — RESERVATION STATUS MACHINE



Diagram — Domain Structure

FLOWCHART — DOMAIN LAYER INTERNALS



Repository Interfaces (Ports)

These are the DIP "ports" — the abstractions use cases depend on. Infrastructure provides the "adapters" (implementations).

INTERFACE	METHODS
IReservationRepository	getAll, getById, create, confirm, cancel, getLogs
IInventoryRepository	getAll, upsert
INameStore	save, resolve, getAll, clear

Domain Events

EVENT CLASS	PUBLISHED BY	PAYLOAD
<code>ReservationCreatedEvent</code>	CreateReservationUseCase	<code>reservation</code> , <code>customerName</code>
<code>ReservationConfirmedEvent</code>	ConfirmReservationUseCase	<code>reservation</code>
<code>ReservationCancelledEvent</code>	CancelReservationUseCase	<code>reservation</code>
<code>InventorySeededEvent</code>	SeedInventoryUseCase	<code>inventory</code>

Domain Exceptions

CLASS	WHEN THROWN
<code>OutOfStockError</code>	Backend returns HTTP 409 on reservation create
<code>InvalidStateError</code>	Backend returns HTTP 409 on confirm / cancel
<code>NotFoundError</code>	Backend returns HTTP 404

Application Layer

Use cases orchestrate the domain. Each class has a single `execute()` method and depends only on domain interfaces — never on `HttpClient` or any React API.

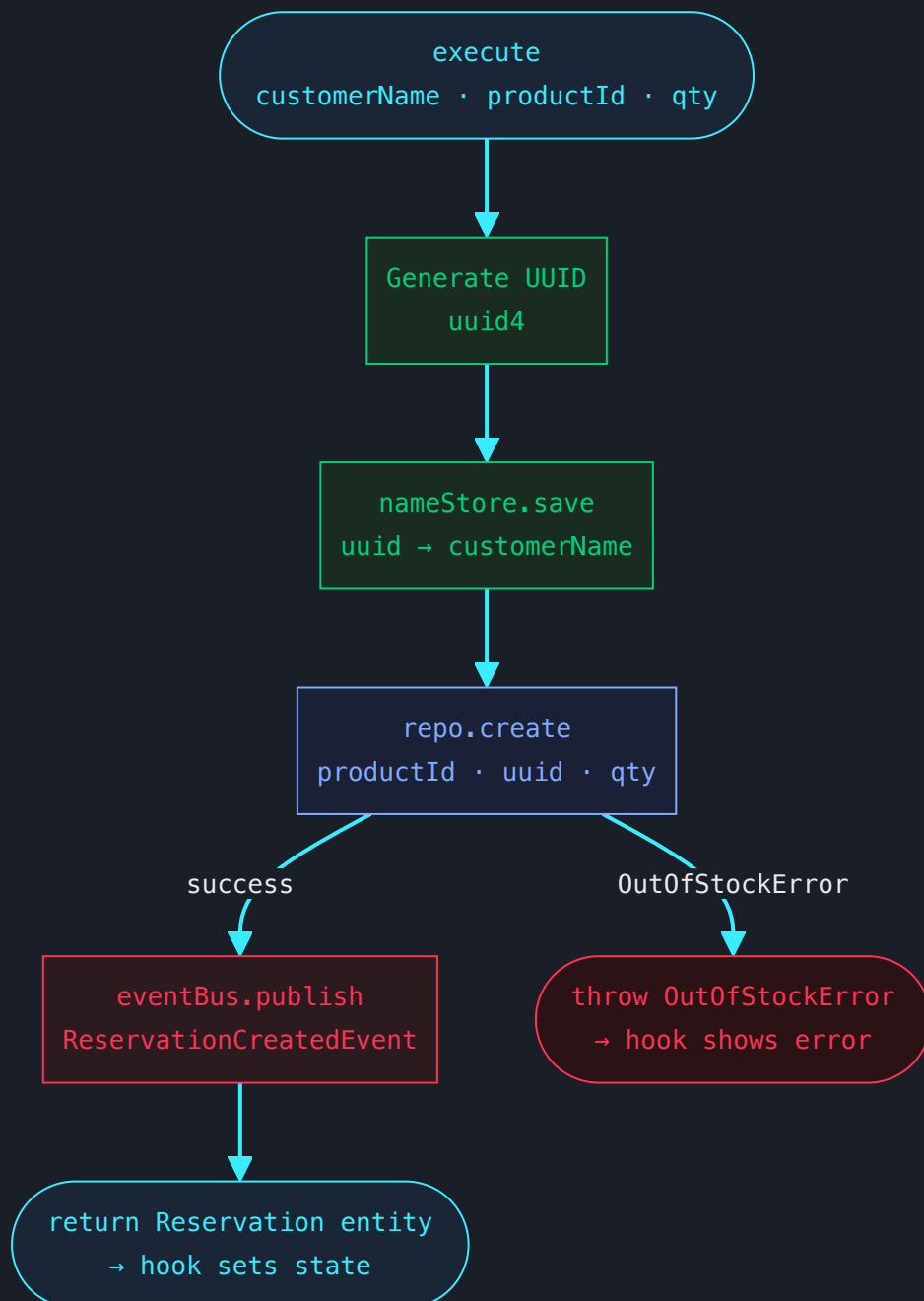
USE CASE	EXECUTE() SIGNATURE	SIDE EFFECTS
<code>CreateReservationUseCase</code>	<code>(customerName, productId, quantity) → Reservation</code>	Saves name to <code>INameStore</code> , publishes <code>ReservationCreatedEvent</code>
<code>ConfirmReservationUseCase</code>	<code>(id) → Reservation</code>	Publishes <code>ReservationConfirmedEvent</code>
<code>CancelReservationUseCase</code>	<code>(id) → Reservation</code>	Publishes <code>ReservationCancelledEvent</code>
<code>GetReservationsUseCase</code>	<code>(params) → PaginatedResult<Reservation></code>	None
<code>GetDashboardStatsUseCase</code>	<code>() → DashboardStats</code>	None

USE CASE	EXECUTE() SIGNATURE	SIDE EFFECTS
<code>SeedInventoryUseCase</code>	<code>(params) → Inventory</code>	Publishes <code>InventorySeededEvent</code>

Diagram — Use Case Orchestration Pattern

Every mutating use case follows the same pattern: validate inputs via domain, call the repository, publish an event. The event bus does the rest.

FLOWCHART — CREATERESERVATIONUSECASE INTERNALS



SimulationService

Provides six named scenarios. Each method accepts a `Logger` callback so log lines stream to the component in real-time.

METHOD	DESCRIPTION
<code>runConcurrentRush</code>	N reservations submitted simultaneously — tests race conditions
<code>runLifecycle</code>	Full happy path: create → confirm for each reservation
<code>runCancelFlow</code>	Creates then immediately cancels
<code>runExpireGuard</code>	Creates reservations and waits for expiry
<code>runBulk</code>	Large-volume create batch
<code>runStress</code>	Mixed concurrent creates + confirms + cancels

Events (EDA)

The Events layer decouples *producers* (use cases that mutate data) from *consumers* (UI side effects like toasts and data refreshes). A use case publishes an event; it has zero knowledge of who is listening.

EventBus

```
// Subscribe – returns an unsubscribe function
const unsubs = eventBus.subscribe<ReservationCreatedEvent>(  
  'ReservationCreated',  
  (e) => console.log(e.reservation.id)  
);  
unsubs(); // call to unsubscribe  
  
// Publish – dispatches to all subscribers synchronously  
eventBus.publish(new ReservationCreatedEvent(reservation, customerName));
```



Singleton pattern

`eventBus` is a module-level singleton exported from `src/events/EventBus.ts`. The DI container imports and passes it to every use case that publishes, ensuring all parts of the app share the same instance.

Handlers

Both handlers are registered once in `AppContext` on mount and automatically unsubscribe on unmount:

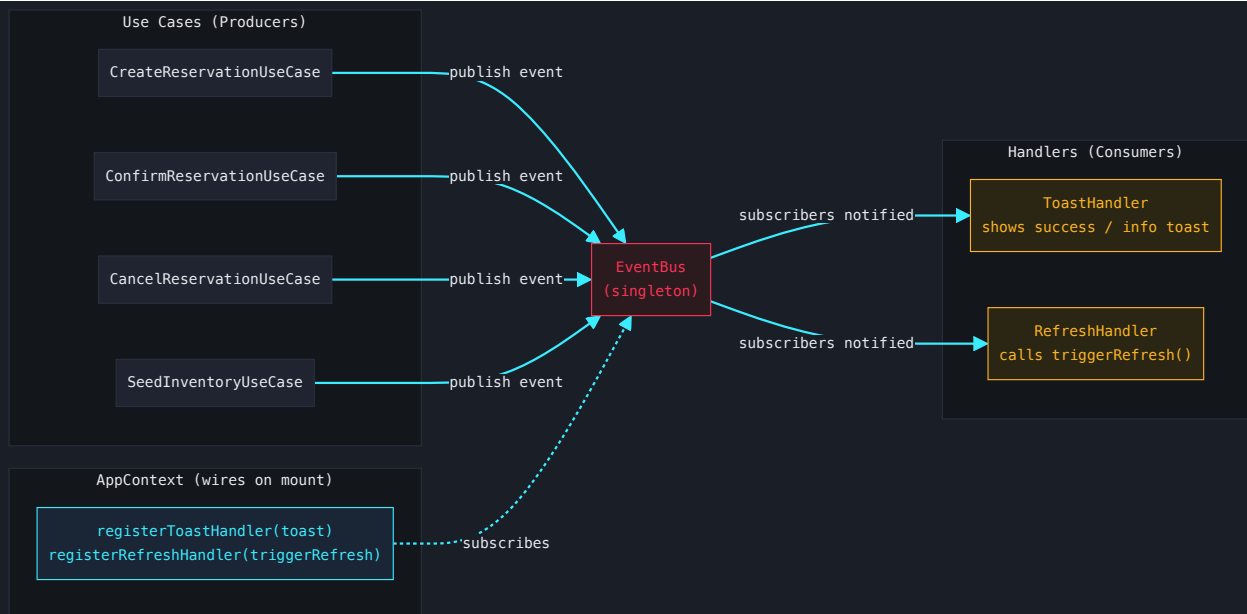
```
// AppContext.tsx – EDA wiring
useEffect(() => {
  const cleanupToast = registerToastHandler(toast);
  const cleanupRefresh = registerRefreshHandler(triggerRefresh);
  return () => { cleanupToast(); cleanupRefresh(); };
}, [toast, triggerRefresh]);
```

To add a new side effect to any existing event — for example, sending an analytics ping when a reservation is confirmed — create a new handler file and register it here. **Never modify the use case that publishes the event.** That is the Open/Closed Principle.

Diagram — EDA Pub/Sub Wiring

Producers and consumers know nothing about each other. The EventBus is the only shared dependency — and it's injected, not imported directly.

FLOWCHART – EVENT BUS PRODUCERS AND CONSUMERS



Infrastructure Layer

Infrastructure adapts the outside world (HTTP API, localStorage) to the domain interfaces. This is the only layer that knows about `fetch`, HTTP status codes, JSON parsing, or `localStorage`.

HttpClient

```

class HttpClient {
  // getBase is a lambda – URL is read fresh on every request.
  // Changes in Settings tab take effect immediately, no reinitialisation.
  constructor(private readonly getBase: () => string) {}

  request<T>(<
    method: string,
    path: string,
    body?: unknown
  >): Promise<{ ok: boolean; status: number; data: T }>
}
  
```

Error mapping in repositories

HTTP STATUS	DOMAIN EXCEPTION THROWN
409 on reservation create	<code>OutOfStockError</code>

HTTP STATUS	DOMAIN EXCEPTION THROWN
409 on confirm / cancel	<code>InvalidStateError</code>
404	<code>NotFoundError</code>
Any other non-2xx	<code>Error</code> with message from response body

LocalStorageNameStore

Implements `INameStore` using the `userNameMap` key in `localStorage`, persisting a `{ [uuid]: fullName }` JSON map across page reloads. The frontend generates a UUID per reservation so names can be resolved even for reservations created directly via the API (fallback: truncated UUID).

DI Container

`src/infrastructure/container.ts` is the **only file** allowed to import both domain interfaces and their concrete implementations simultaneously. It wires everything together once at module load time:

```
// container.ts – simplified
const httpClient = new HttpClient(() => localStorage.getItem('api_base') ?? 'ht
const reservationRepo = new ApiReservationRepository(httpClient);
const inventoryRepo = new ApiInventoryRepository(httpClient);
const nameStore = new LocalStorageNameStore();

export const container = {
  createReservation: new CreateReservationUseCase(reservationRepo, nameStore, even
  confirmReservation: new ConfirmReservationUseCase(reservationRepo, eventBus),
  // ... all 6 use cases + SimulationService
  nameStore, // exposed for name resolution in components
};

// Health check helper used by AppContext
export function checkHealth(): Promise<boolean>
```

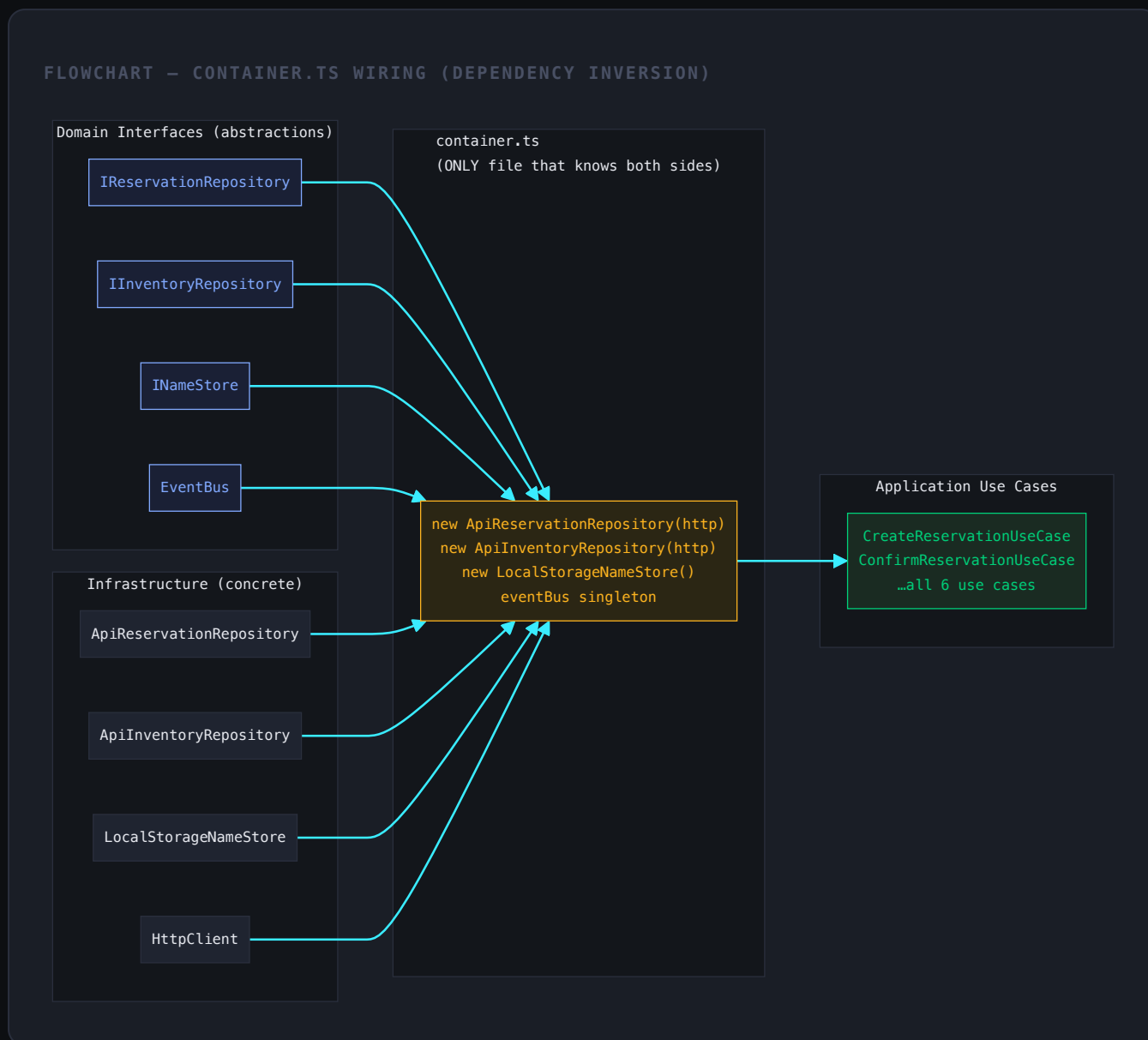


Swapping implementations

To use an in-memory repository (for testing), replace `ApiReservationRepository` with `InMemoryReservationRepository` in `container.ts`. Every use case, hook, and component automatically picks up the new implementation — nothing else changes.

Diagram — DI Container Wiring

`container.ts` is the only file that sees both sides — domain interfaces on the left, concrete implementations on the right. Everything else stays in its own layer.



Presentation Hooks

Hooks bridge use cases to React state. Each hook calls use cases, manages `loading` / `error` state, and returns everything a component needs. **Components call hooks only — they never import from `container` directly** (the exception: `container.nameStore` for name resolution is allowed in components).

HOOK	KEY RETURNS	RE-FETCHES WHEN
<code>useDashboard()</code>	<code>stats</code> , <code>loading</code> , <code>error</code>	<code>refreshTick</code> changes
<code>useReservations()</code>	<code>result</code> , <code>loading</code> , <code>confirm</code> , <code>cancel</code> , <code>getLogs</code> , <code>page</code> , <code>filter</code>	<code>refreshTick</code> , <code>page</code> , <code>filter</code>
<code>useCreateReservation()</code>	<code>execute</code> , <code>loading</code> , <code>error</code>	—
<code>useSimulate()</code>	<code>run</code> , <code>running</code> , <code>logs</code> , <code>clearLogs</code>	—
<code>useInventory()</code>	<code>items</code> , <code>loading</code> , <code>seed</code> , <code>seeding</code> , <code>seedError</code>	<code>refreshTick</code>

Cancellable fetch pattern

All fetching hooks use this pattern to prevent state updates on unmounted components:

```
useEffect(() => {
  let cancelled = false;
  setLoading(true);

  container.getReservations.execute(params)
    .then((r) => { if (!cancelled) setResult(r); })
    .catch((e: Error) => { if (!cancelled) setError(e.message); })
    .finally(() => { if (!cancelled) setLoading(false); });

  return () => { cancelled = true; };
}, [refreshTick, page, filter]);
```

Components

Components are pure presentation. They render state returned by hooks and call hook action functions in response to user interactions. No business logic lives here.

Shared components

COMPONENT	PROPS	NOTES
<code><ExpiryBar></code>	<code>reservation:</code> <code>Reservation</code>	Auto-ticks every 1s. Colour transitions green → yellow → red as expiry approaches. Delegates all math to entity methods.
<code><Badge></code>	<code>status:</code> <code>ReservationStatus</code>	Reads label and colour from <code>ReservationStatusMeta</code> . Pulsing dot for ACTIVE status.
<code><Modal></code>	<code>open</code> , <code>onClose</code> , <code>title</code> , <code>children</code>	Backdrop click closes. Used for the reservation status-change log.
<code><Spinner></code>	<code>size?</code> , <code>color?</code>	CSS animation spinner; no library.
<code><Toast></code>	Consumed by <code>ToastStack</code>	Auto-dismiss after 3.5 s. Types: success / error / info.

Layout components

COMPONENT	RESPONSIBILITY
<code><Sidebar></code>	Navigation tabs with active indicator, LIVE/OFFLINE pulsing dot, version footer. Reads <code>tab</code> and <code>online</code> from <code>useApp()</code> .
<code><TopBar></code>	Breadcrumb title, live HH:MM:SS clock (ticks every second via its own interval), LIVE/OFFLINE pill, manual refresh button.

Customer name resolution

Reservations use a UUID `userId`. Names are stored in localStorage and resolved at render time:

```
// In any component that renders a customer name:
const name = container.nameStore.resolve(r.userId)
  ?? r.userId.slice(0, 8) + '...';
// Falls back to truncated UUID for reservations created via external API
```

AppContext API

All components access global state via `useApp()`. Must be called inside `<AppProvider>`, which wraps the entire app in `main.tsx`.

VALUE	TYPE	DESCRIPTION
<code>tab</code>	<code>TabId</code>	Currently active tab
<code>setTab</code>	<code>(t: TabId) => void</code>	Navigate to a tab programmatically
<code>apiBase</code>	<code>string</code>	Backend base URL; persisted to <code>localStorage</code>
<code>setApiBase</code>	<code>(url: string) => void</code>	Updates URL; trailing slash stripped automatically
<code>online</code>	<code>boolean</code>	<code>true</code> when backend health check passes (polled every 10 s)
<code>toast</code>	<code>(msg, type?) => void</code>	Show a notification. type: <code>'success' 'error' 'info'</code>
<code>toasts</code>	<code>Toast[]</code>	Active toast queue (consumed by <code>ToastStack</code> in <code>App.tsx</code>)
<code>dismissToast</code>	<code>(id: number) => void</code>	Manually remove a toast
<code>refreshTick</code>	<code>number</code>	Increments every 10 s <i>and</i> after every mutating domain event. Use as a <code>useEffect</code> dependency to auto-refresh data.
<code>triggerRefresh</code>	<code>() => void</code>	Manually force a refresh tick (e.g. from the TopBar refresh button)

localStorage keys

KEY	VALUE TYPE	MANAGED BY
<code>api_base</code>	<code>string</code>	<code>AppContext.setApiBase()</code>
<code>userNameMap</code>	<code>{ [uuid]: fullName }</code> JSON	<code>LocalStorageNameStore</code>

`toast()` is called by EDA handlers, not components

In the DDD/EDA architecture, toast notifications are triggered by `ToastHandler` reacting to domain events — not called directly in component event handlers. The exception is error paths inside hooks where a domain exception (e.g. `OutOfStockError`) is caught and surfaced to the user.

Design System



No CSS libraries

No Tailwind, no styled-components, no CSS modules. All styling uses the CSS custom properties in `index.css`, applied via inline styles or the utility class names below.

Colour tokens

TOKEN	HEX	USE FOR
<code>--bg-base</code>	#0d0f12	Page background
<code>--bg-surface</code>	#13161b	Sidebar, panels
<code>--bg-card</code>	#1a1e26	Cards, code blocks
<code>--bg-card2</code>	#1f2430	Table headers, elevated cards
<code>--border</code>	#2a2f3d	All borders and dividers
<code>--accent</code>	#3aecff	Primary accent, active states, links
<code>--accent-dim</code>	rgba(58,236,255,0.12)	Active state backgrounds
<code>--green</code>	#00d17a	ACTIVE status, success
<code>--yellow</code>	#ffb520	CANCELLED status, warnings
<code>--red</code>	#ff3355	EXPIRED status, errors
<code>--purple</code>	#a78bfa	CONFIRMED status
<code>--text</code>	#e8eaf0	Primary body text
<code>--text-muted</code>	#7c8494	Secondary text
<code>--text-dim</code>	#4a5165	Placeholders, low-contrast labels

Typography tokens

TOKEN	FONT	USE FOR
<code>--font-display</code>	Syne	Headings, sidebar labels, brand text

TOKEN	FONT	USE FOR
<code>--font-body</code>	DM Sans	Body text, paragraphs, table cells
<code>--font-mono</code>	JetBrains Mono	Code, IDs, timestamps, stat values

Utility class names

CLASS	DESCRIPTION
<code>card</code>	Surface card with border and border-radius
<code>btn</code>	Base button reset
<code>btn-primary</code>	Accent-coloured filled button
<code>btn-sm</code>	Smaller button variant
<code>btn-success</code>	Green confirm button
<code>btn-danger</code>	Red cancel/delete button
<code>fade-in</code>	Opacity 0 → 1, 0.25 s
<code>slide-in</code>	translateY 12px → 0, 0.25 s
<code>dot-grid</code>	Dot-pattern background (main content wrapper)

Pulsing dot

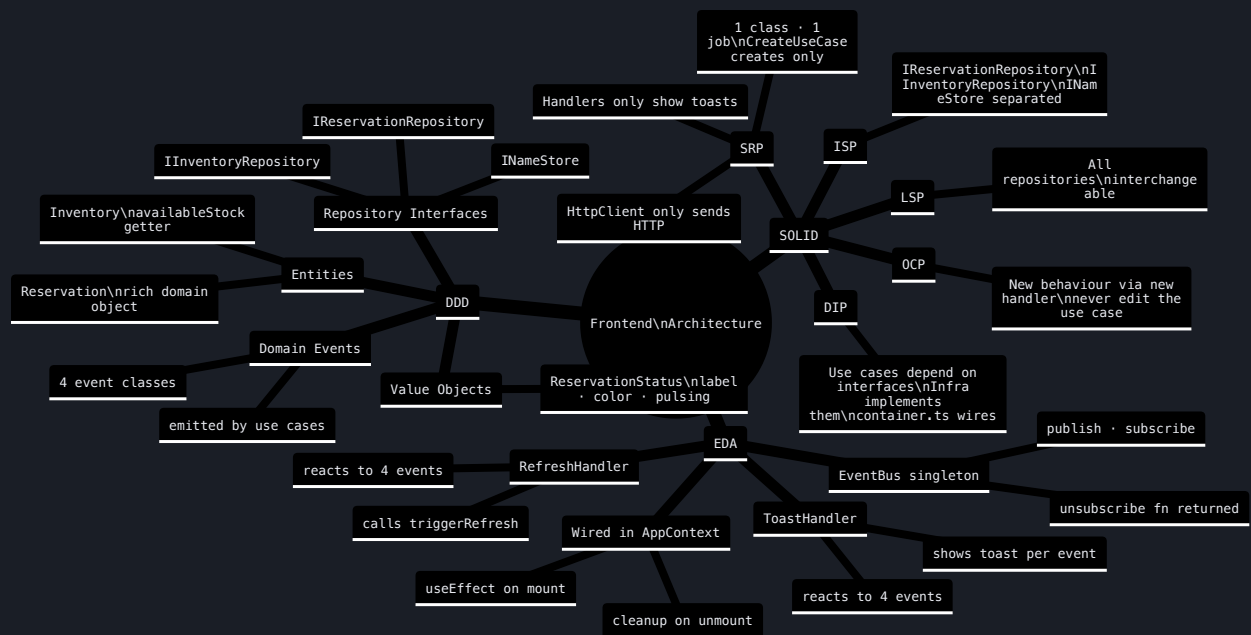
```
// LIVE indicator and ACTIVE badge – apply via inline style:
<div style={{ animation: 'blink 1.4s ease-in-out infinite' }} />
```

DDD · SOLID · EDA in This Codebase

These patterns are not just concepts — every module maps directly to one or more of them.

Diagram — DDD · SOLID · EDA Concept Map

MINDMAP – PATTERNS APPLIED IN THIS CODEBASE



DDD

Domain-Driven Design

Business rules live inside `Reservation` and `Inventory` entities (`canConfirm`, `expiryPercentElapsed`). Repository interfaces are the domain "ports". Components never recompute what the entity already knows.

SRP

Single Responsibility

Each class has one job: `CreateReservationUseCase` creates. `ToastHandler` shows toasts. `HttpClient` sends requests. A change to toast wording never touches the use case.

DIP

Dependency Inversion

Use cases depend on `IReservationRepository` (interface), not `ApiReservationRepository` (class). `container.ts` is the only file that closes this gap. Components depend on hooks, not the container.

OCP

Open/Closed

Add a new reaction to `ReservationCreated` (e.g. analytics ping) by subscribing a new handler. The use case that publishes is *never* edited. Existing behaviour is preserved by default.

EDA

Event-Driven Architecture

After any mutation, use cases publish a `DomainEvent` to the `EventBus` singleton.

`ToastHandler` and `RefreshHandler` react independently. Neither the use case nor the component knows what happens after the event is published — they only fulfil their own responsibilities.

How-To Recipes

Add a new tab

1

Add the TabId value

In `src/types/index.ts`, add your new ID (e.g. `'analytics'`) to the `TabId` union type.

2

Create the component

Create `src/components/Analytics/Analytics.tsx`. Consume a presentation hook. No direct API calls in the component.

3

Register in Sidebar

Add an entry to the `NAV` array in `Sidebar.tsx` with your `id`, `label`, and icon character.

4

Register in TopBar

Add your tab's title and description to the maps in `TopBar.tsx`.

5

Mount in App.tsx

Add a render branch for `tab === 'analytics'` in the conditional block in `App.tsx`.

Add a new API operation (end-to-end)

1

Add the method to the repository interface

Declare the signature in `src/domain/interfaces/IReservationRepository.ts`. This is the contract everything else depends on.

2

Implement it in the API repository

Add the HTTP call in `ApiReservationRepository.ts`. Map HTTP response to a domain entity; map error codes to domain exceptions.

3

Create a use case

Create `src/application/use-cases/MyUseCase.ts`. It calls the interface. If it causes side effects, publish a domain event at the end of `execute()`.

4

Wire it in container.ts

Add `myOp: new MyUseCase(reservationRepo, eventBus)` to the `container` export.

5

Create a presentation hook

Call `container.myOp.execute()` from a hook. Expose loading, error, and result. The component imports only the hook.

Add a new domain event side effect

1

Create a handler file

Create `src/events/handlers/MyHandler.ts`. Export `registerMyHandler(deps)` that calls `eventBus.subscribe()` and returns a cleanup function.

2

Register in AppContext

Call `registerMyHandler()` inside the EDA `useEffect` in `AppContext.tsx` alongside the existing handlers. Include the cleanup in the return function.

Change the backend URL at runtime

Navigate to the **Settings** tab and update the API Base URL field. The change persists to `localStorage` and `HttpClient` reads it fresh on every request — no page reload required.

Conventions & Rules

TypeScript rules

RULE	DETAIL
Strict mode always on	<code>noImplicitAny</code> , <code>strictNullChecks</code> , <code>noUnusedLocals</code> — all enforced at build time
No <code>any</code>	Use <code>unknown</code> + a type guard instead. If truly necessary, add a comment explaining why.
No <code>ts-ignore</code>	Fix the type error — do not suppress it
Explicit callback parameter types	<code>.map((r: Reservation) => ...)</code> — implicit <code>any</code> in callbacks is a build error
Remove unused imports	<code>noUnusedLocals</code> is on — unused imports are compile errors

Architecture rules

RULE	WHY
Components do not call use cases or HttpClient directly	Must go through a hook — maintains testability and SRP
Domain layer has zero external imports	No React, no fetch, no localStorage — must be portable and framework-agnostic
Only <code>container.ts</code> imports both interfaces and implementations	Maintains the DIP boundary throughout the rest of the codebase
New side effects go in handler files, not use cases	Preserves OCP — use cases stay closed for modification
All HTTP through <code>HttpClient.request()</code>	Never call <code>fetch()</code> directly — error normalisation and base URL logic live there

Styling rules

RULE
No CSS libraries (Tailwind, styled-components, CSS modules). Use CSS variables + inline styles.
Use utility classes (<code>card</code> , <code>btn</code> , <code>btn-primary</code> , etc.) for base styles; extend with inline styles.
Do not rename or change the colour palette or font stack — they are referenced across every component.

RULE

Do not introduce a router library — the tab model is intentionally simple.

Do not add a state management library (Redux, Zustand, Jotai).

Build gate



npm run build must exit 0 before any commit

Runs `tsc` (full strict TypeScript) then `vite build`. Common causes of failure: unused imports, implicit `any` in callbacks, incorrect type annotation on a `setState` updater function, or a `keyof (T | null)` expression resolving to `never`.

inventory-reservation-client · DDD + SOLID + EDA · React 18 · TypeScript 5.5 · Vite 6